COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	8/4/2026	**DATE SUBMITTED:**	
NAME:	Estorco, Clint Hoven A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Hands-on Activity 4.1 STACKS

1. PROCEDURE OUTPUT

Main.cpp

```cpp
#include <iostream>
#include "stackList.h"

int main(){

    std::cout << "--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---\n";
    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    Top<char>(); // Top should be 'e'
    displayALL<char>();
    std::cout << "\n------ Popping all the elements -----\n";

    while(!isEmpty<char>()){
        std::cout << "Popped: " << pop<char>() << std::endl;
    }

    std::cout << "\n------ Testing the Underflow -----\n";
    pop<char>();

    return 0;
}
```

stackList.h

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

#include <iostream>

// Node structure
template <typename T>
struct Node {
    T data;
    Node* next;
};

// Encapsulating the stack inside a class template
template <typename T>
class Stack {
private:
    Node<T>* head;

public:
    Stack() {
        head = NULL;
    }

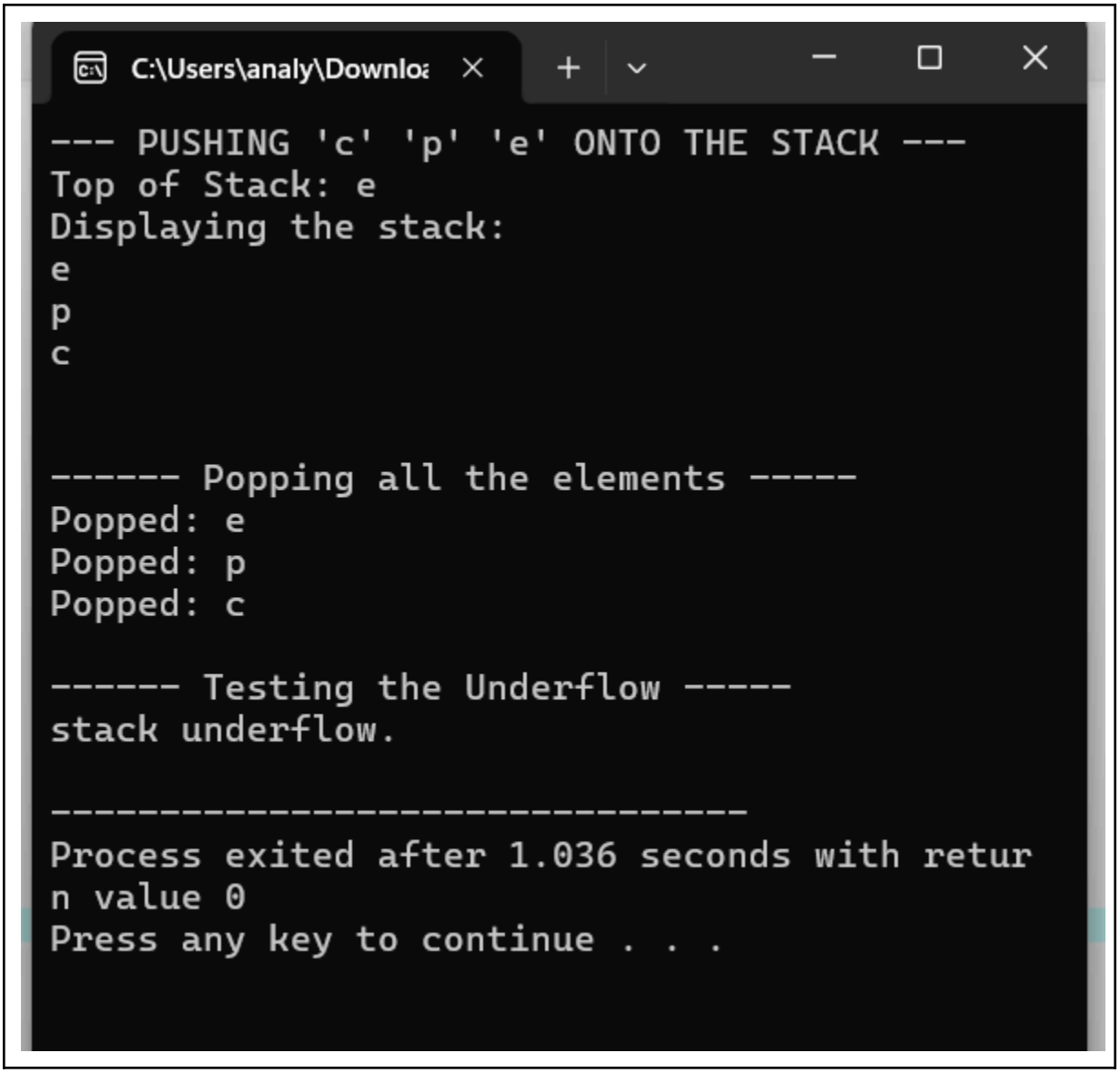
    void push(T newData) {
        Node<T>* newNode = new Node<T>;
        newNode->data = newData;
        newNode->next = head;
        head = newNode;
    }

    T pop() {
        if (isEmpty()) {
            std::cout << "stack underflow." << std::endl;
            return T();
        }
        Node<T>* temp = head;
        T tempVal = temp->data;
        head = head->next;
        delete temp;
        return tempVal;
    }

    bool isEmpty() {
        return head == NULL;
```

```
45      }
46
47      void Top() {
48          if (isEmpty()) {
49              std::cout << "Stack underflow" << std::endl;
50              return;
51          }
52          std::cout << "Top of Stack: " << head->data << std::endl;
53      }
54
55      void displayALL() {
56          if (isEmpty()) {
57              std::cout << "stack underflow\n";
58              return;
59          }
60          std::cout << "Displaying the stack: \n";
61          Node<T>* temp = head;
62          while (temp != NULL) {
63              std::cout << temp->data << "\n";
64              temp = temp->next;
65          }
66          std::cout << "\n";
67      }
68  };
69
70  #endif
```

OUTPUT:



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

INPUT	OUTPUT

```cpp
#include <iostream>
#include <string>
#include "StackList.h"

bool isPairMatching(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

bool checkBalanced(const std::string& expr) {
    ArrayStack stack;

    for (size_t i = 0; i < expr.length(); ++i) {
        char ch = expr[i];

        if (ch == '(' || ch == '{' || ch == '[') {
            stack.push(ch);
        } else if (ch == ')' || ch == '}' || ch == ']') {
            if (stack.isEmpty()) {
                std::cout << "Error: Unmatched closing symbol '" << ch << "' at index " << i << "\n";
                return false;
            }

            char topChar = stack.pop();
            if (!isPairMatching(topChar, ch)) {
                std::cout << "Error: Mismatched symbols '" << topChar << "' and '" << ch << "' at index " << i << "\n";
                return false;
            }
        }
    }
```

```cpp
    }

    if (!stack.isEmpty()) {
        std::cout << "Error: Expression has unclosed opening symbols.\n";
        return false;
    }

    return true;
}

int main() {
    std::string inputExpr;
    std::cout << "  BALANCED SYMBOL CHECKER (ARRAY STACK)    \n";
    std::cout << "Enter expression: ";
    std::getline(std::cin, inputExpr);

    if (checkBalanced(inputExpr)) {
        std::cout << "\nResult: BALANCED\n";
    } else {
        std::cout << "\nResult: NOT BALANCED\n";
    }

    return 0;
}
```

```
C:\msys64\ucrt64\bin>"C:\Users\analy\OneDrive\Desktop\STACKS\stack main.exe"
  BALANCED SYMBOL CHECKER (ARRAY STACK)
Enter expression: %

Result: BALANCED
```

StackList.h > ...

```
#ifndef STACK_LIST_H
#define STACK_LIST_H

#include <iostream>

#define MAX_SIZE 100

class ArrayStack {
private:
    char data[MAX_SIZE];
    int topIndex;

public:
    ArrayStack() {
        topIndex = -1;
    }

    bool push(char value) {
        if (topIndex >= MAX_SIZE - 1) {
            std::cout << "Stack overflow!\n";
            return false;
        }
        data[++topIndex] = value;
        return true;
    }

    char pop() {
        if (isEmpty()) {
            return '\0';
        }
        return data[topIndex--];
    }

    char peek() const {
        if (isEmpty()) {
            return '\0';
        }
        return data[topIndex];
    }

    bool isEmpty() const {
        return topIndex == -1;
    }
};
```

```
43        }
44    };
45
46    #endif
```

INPUT	OUTPUT

```cpp
#include <iostream>
#include <string>
#include "StacksLinkedList.h"

bool isPairMatching(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

bool checkBalanced(const std::string& expr) {
    LinkedListStack stack;

    for (size_t i = 0; i < expr.length(); ++i) {
        char ch = expr[i];

        if (ch == '(' || ch == '{' || ch == '[') {
            stack.push(ch);
        } else if (ch == ')' || ch == '}' || ch == ']') {
            if (stack.isEmpty()) {
                std::cout << "Error: Unmatched closing symbol '" << ch << "' at index " << i << "\n";
                return false;
            }

            char topChar = stack.pop();
            if (!isPairMatching(topChar, ch)) {
                std::cout << "Error: Mismatched symbols '" << topChar << "' and '" << ch << "' at index " << i << "\n";
                return false;
            }
        }
    }

    if (!stack.isEmpty()) {
        std::cout << "Error: Expression has unclosed opening symbols.\n";
        return false;
    }

    return true;
}

int main() {
    std::string inputExpr;

    std::cout << "  BALANCED SYMBOL CHECKER (LINKED LIST)    \n";
```

```cpp
    std::cout << "Enter expression: ";
    std::getline(std::cin, inputExpr);

    if (checkBalanced(inputExpr)) {
        std::cout << "\nResult: BALANCED\n";
    } else {
        std::cout << "\nResult: NOT BALANCED\n";
    }

    return 0;
}
```

```
C:\msys64\ucrt64\bin>"C:\Users\analy\OneDrive\Desktop\STACKS\Stack linked list main.exe"
  BALANCED SYMBOL CHECKER (LINKED LIST)
Enter expression: *

Result: BALANCED

C:\msys64\ucrt64\bin>
```

StacksLinkedList.h > ...

```h
#ifndef STACKS_LINKED_LIST_H
#define STACKS_LINKED_LIST_H

#include <iostream>

struct Node {
    char value;
    Node* next;
};

class LinkedListStack {
private:
    Node* topNode;

public:
    LinkedListStack() {
        topNode = nullptr;
    }

    ~LinkedListStack() {
        while (!isEmpty()) {
            pop();
        }
    }

    void push(char val) {
        Node* newNode = new Node();
        newNode->value = val;
        newNode->next = topNode;
        topNode = newNode;
    }

    char pop() {
        if (isEmpty()) {
            return '\0';
        }
        Node* temp = topNode;
        char poppedValue = temp->value;
        topNode = topNode->next;
        delete temp;
        return poppedValue;
    }

    char peek() const {
        if (isEmpty()) {
            return '\0';
        }
        return topNode->value;
    }

    bool isEmpty() const {
        return topNode == nullptr;
    }
};

#endif
```

StacksLinkedList.h > ...

```h
#ifndef STACKS_LINKED_LIST_H
#define STACKS_LINKED_LIST_H

#include <iostream>

struct Node {
    char value;
    Node* next;
};

class LinkedListStack {
private:
    Node* topNode;

public:
    LinkedListStack() {
        topNode = nullptr;
    }

    ~LinkedListStack() {
        while (!isEmpty()) {
            pop();
        }
    }

    void push(char val) {
        Node* newNode = new Node();
        newNode->value = val;
        newNode->next = topNode;
        topNode = newNode;
    }

    char pop() {
        if (isEmpty()) {
            return '\0';
        }
        Node* temp = topNode;
        char poppedValue = temp->value;
        topNode = topNode->next;
        delete temp;
        return poppedValue;
    }

```

CONCLUSION:

The Stack Abstract Data Type (ADT) was successfully implemented in this hands-on activity using both a linked list and Array method in C++. All five core operations (push, pop, peek, isEmpty, and queue underflow errors) were fully tested for Last-In, First-Out (LIFO) operation and valid exception handling, namely when the stack underflow's cases have been dealt.All five major operations (push, pop, peek, isEmpty, and queue underflow error cases) were tested extensively to ensure correct behavior and to confirm Last-In, First-Out (LIFO) operation of the stack and valid exception handling (when queue underflow's cases have been taken care). Further, both stack types have been used to create a balanced symbol checker that checks whether an expression has the matching of delimiters. One of the differences in how these two types of stacks were implemented was the use of fixed memory allocation for the array-based stack, whereas the linked-list approach was flexible with memory allocation, allowing it to expand and contract as needed.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.